IC-272 — Introduction to Machine Learning

Lab 6 — k-Nearest Neighbours and Cross-Validation

Indian Institute of Technology Mandi

Total marks: 100 — Duration: 2 hours

Marks: 20 for code, 80 for the viva. There are three tasks. Your TA takes a short viva on each task *as soon as you finish it* — do not save them all for the end.

There is also a homework section at the end. It will not be vivaed and carries no marks today. It is examinable in the final lab exam.

You will reuse two functions you have already written: `pairwise_distances` from Lab 1 and `standardise` from Lab 5. Copy them into a cell at the top. If either was wrong, fix it first — today they are tools, not exercises.

Rules

- Permitted: `numpy`, `pandas`, `matplotlib`. `scikit-learn` **is not permitted** — a task that calls it scores zero for that task.
- Barred: `scipy.spatial`, `np.argpartition` is allowed but `sklearn.neighbors` is not.
- No loop over the query rows in Task 1. Use your Lab 1 distance matrix.

Setting up — type this first

`ic272_lab6_vibration.csv` holds 400 machines. `vib_x` and `vib_y` are vibration deviations on two axes in mm/s; `machine_hours` is hours since the last overhaul; `faulty` is the label, 1 for a machine needing service.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("ic272_lab6_vibration.csv")
features = ["vib_x", "vib_y", "machine_hours"]
X = df[features].to_numpy(float)
y = df["faulty"].to_numpy()

idx = np.random.default_rng(0).permutation(len(y))
te, tr = idx[:100], idx[100:]
X_train, y_train = X[tr], y[tr]
X_test,  y_test  = X[te], y[te]

print("train:", X_train.shape, " test:", X_test.shape)   # (300, 3)  (100, 3)
```

Use `seed = 0` exactly as written, or your numbers will not match anyone else's. The classes are close to balanced, so the majority-class baseline on the test set is 0.55 — every accuracy you report today has to be judged against that, not against zero.

Why this lab exists — read this, then run it

Two things about this dataset decide the whole lab. Find both yourself.

First: the classes are not linearly separable

Scatter `vib_y` against `vib_x`, coloured by `faulty`. Normal machines sit near zero deviation on both axes. Faulty machines deviate — but in *some* direction, not a particular one, so they form a ring around the normal cluster.

No straight line separates a ring from its centre. The linear models of Labs 4 and 5 cannot do this at all, whatever you set λ to. k-nearest neighbours draws no line: it looks at whatever is nearby and takes a vote, so a curved boundary costs it nothing.

Second: one column will eat your distance function

Print the standard deviation of each column of the training set:

```
for name, s in zip(features, X_train.std(axis=0)):
    print(f"{name:15s}{s:12.3f}")
```

column	training sd
`vib_x`	1.733
`vib_y`	1.736
`machine_hours`	14590.484

Euclidean distance squares the difference in every column and adds them up. A difference of 20,000 hours squares to 4×10^8; a difference of 3 mm/s squares to 9. Take any test machine and ask what share of its squared distance to the training set comes from each column: `machine_hours` contributes **99.99999**%, and the two vibration axes together contribute about 3×10^{-6} per cent.

So an unstandardised nearest-neighbour search on this data is not using vibration at all. It is ranking machines by overhaul hours, which is unrelated to the fault. Task 2 measures what that costs you.

Task 1 — A k-NN classifier [code 8 — viva 24]

(a) Write `knn_predict(X_train, y_train, X_query, k)` returning one predicted label per row of `X_query`. Use your Lab 1 `pairwise_distances` to get the full query-by-training distance matrix, take the k smallest distances in each row, and let those k neighbours vote. No loop over the query rows. **[6]**

(b) Sanity-check it: predict on the *training* rows themselves with $k = 1$. **[2]**

`np.argsort` along `axis=1` gives you the neighbour indices in order of distance; the first k columns are the nearest k. For a two-class vote, the mean of the neighbours' labels exceeding 0.5 is the majority.

In (b) every prediction must come back exactly equal to the true label. A point's own nearest neighbour is itself, at distance zero. If you do not get a perfect match, your indexing is wrong — fix it before moving on, because Tasks 2 and 3 are built on this function.

Task 2 — What standardisation is worth [code 4 — viva 24]

(a) Using your `standardise` from Lab 5, produce `Z_train` and `Z_test`. Remember μ and σ come from the training set only. **[2]**

(b) Report the test accuracy of `knn_predict` at $k = 7$ twice: once on the raw features, once on the standardised ones. **[2]**

Compare both numbers against the 0.55 majority baseline before you call your TA. One of them does not beat it. Be ready to say which, and why that is the expected result rather than a bug.

Task 3 — Choosing k by cross-validation [code 8 — viva 32]

k is a choice, not something the data hands you. Too small and each prediction follows a single possibly-mislabelled neighbour; too large and the vote reaches across the boundary and drowns the local structure. As in Lab 5, the honest way to choose is cross-validation on the training rows alone.

(a) Write `k_fold_cv_knn(X, y, k, folds=5, seed=0)` returning the mean validation *accuracy* across the folds. This is your Lab 5 `k_fold_cv` with ridge swapped for `knn_predict` and RMSE swapped for accuracy. **[5]**

(b) Evaluate it on the standardised training set for $k \in \{1, 3, 5, 7, 9, 11, 15, 21, 31, 51, 75\}$, print a table of k against CV accuracy, and report the best k. **[2]**

(c) Report the test accuracy at your chosen k, and also at $k = 1$ and $k = 75$. **[1]**

Plot the curve if it helps you read it — k on a logarithmic x-axis. You are looking for a rise, a peak and a fall. Both ends of that curve are a failure mode with a name, and you will be asked for both.

A warning about (a): the fold you hold out must not appear in the data you pass as `X_train` to `knn_predict`. If it does, every held-out point finds itself as its own nearest neighbour and your CV accuracy will be far too high.

Homework — not vivaed today, examinable in the final lab exam

H1. The curse of dimensionality. Keep only `vib_x` and `vib_y`, then append e columns of pure `np.random.normal(0, 1)` noise for $e \in \{0, 2, 5, 10, 20, 50\}$. Standardise, and for each e record the test accuracy at $k = 7$ and the ratio $\min(D)/\max(D)$ over the test-to-train distance matrix. Plot both against e.

The accuracy falls and the ratio climbs towards 1. Explain the connection: when the nearest point is barely nearer than the furthest, "nearest neighbour" stops carrying information. This is the effect from the lecture, measured on your own data.

H2. Distance-weighted voting. Instead of a plain majority, weight each neighbour's vote by $1/(d + \varepsilon)$ with a small ε to avoid division by zero. Redo the sweep over k. Does the fall-off at large k become gentler, and why would you expect that?

H3. Beyond accuracy. At your chosen k, build the confusion matrix with your own Lab 1 function and report precision, recall and F_1 for the faulty class. Accuracy alone was enough here only because the classes are balanced — say what you would have reported instead if 95% of machines were normal.

H4. The decision boundary. Build a grid over the `vib_x`–`vib_y` plane with `np.meshgrid`, classify every grid point at $k = 1$ and at your chosen k, and show the two predicted regions with `imshow` or `contourf`. Put the training points on top. The $k = 1$ boundary is ragged and encloses single points; the larger-k boundary is smooth. That picture is what "overfitting" means for this model.

Submission

- Call your TA as you finish each task. All three vivas must be done before you leave.
- Keep the notebook; the homework goes in the same file.